

# UART-Driven 4x4 Crossbar Packet Switch Design

## *RTL Implementation Using Verilog HDL*

---

NTI - National Telecommunication Institute  
MCIT - Ministry of Communications and Information Technology

FPGA Flow & RTL Design with Verilog Training Program

### This Project Was Developed By:

Mohamed Ahmed Mahmoud

Mariam Badawy Helmy

Sama Mohamed Ahmed

Mahmoud Mostafa Elsayed

Abanoub Ashraf Saneed

Under The Supervision Of: **Eng. Mohamed Salah**

## Table of Contents

---

*Table of contents updated to match the reorganized report.*

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 1. Introduction .....                                           | 4  |
| 1.1 Project Objectives .....                                    | 4  |
| 1.2 Design Tools and Language .....                             | 5  |
| 2. System Architecture .....                                    | 6  |
| 2.1 Architecture 1 — Standalone Packet Switch .....             | 6  |
| 2.2 Architecture 2 — Target UART-Integrated Packet Switch ..... | 7  |
| 2.3 Key Design Decisions .....                                  | 8  |
| 3. Packet Format .....                                          | 10 |
| 3.1 Packet Field Definitions .....                              | 10 |
| 3.2 Packet Type Encoding .....                                  | 10 |
| 3.3 Priority Encoding .....                                     | 11 |
| 3.4 Shared Header Definition File .....                         | 11 |
| 4. Module Descriptions .....                                    | 12 |
| 4.1 FIFO Queue (fifo_queue.v) .....                             | 12 |
| 4.1.1 Internal Structure .....                                  | 12 |
| 4.1.2 First-Word Fall-Through .....                             | 12 |
| 4.1.3 Port Interface .....                                      | 13 |
| 4.1.4 Parameters .....                                          | 13 |
| 4.2 Header Parser (header_parser.v) .....                       | 13 |
| 4.2.1 Bit Slicing .....                                         | 14 |
| 4.2.2 Port Interface .....                                      | 14 |
| 4.3 Routing Table (routing_table.v) .....                       | 14 |
| 4.3.1 Entry Format .....                                        | 15 |
| 4.3.2 Important Initialisation Note .....                       | 15 |
| 4.3.3 Port Interface .....                                      | 16 |
| 4.4 Round-Robin Arbiter (arbiter.v) .....                       | 16 |
| 4.4.1 Round-Robin Algorithm .....                               | 16 |
| 4.4.2 One-Hot Grant .....                                       | 16 |
| 4.4.3 Step-by-Step Operation .....                              | 17 |
| 4.4.4 Port Interface .....                                      | 17 |



|                                                                |    |
|----------------------------------------------------------------|----|
| 4.5 Crossbar Switch Fabric (crossbar.v) .....                  | 18 |
| 4.5.1 Flat Bus Interface .....                                 | 18 |
| 4.5.2 Port Interface .....                                     | 19 |
| 4.5.3 Default Output Clearing .....                            | 19 |
| 4.6 Top-Level Module (packet_switch_top.v) .....               | 20 |
| 4.6.1 Generate Blocks .....                                    | 20 |
| 4.6.2 Back-Pressure .....                                      | 20 |
| 4.6.3 Flat Bus Wiring .....                                    | 20 |
| 5. Testbench and Verification .....                            | 21 |
| 5.1 Testbench Strategy .....                                   | 21 |
| 5.2 Packet Switch Testbench Details .....                      | 21 |
| 5.3 Known Bugs Identified and Fixed During Verification .....  | 22 |
| 5.4 Simulation Waveforms .....                                 | 22 |
| 5.4.1 Test Case 1 .....                                        | 22 |
| 5.4.2 Test Case 2 .....                                        | 23 |
| 5.4.3 Test Case 3 .....                                        | 24 |
| 5.4.4 Test Case 4 .....                                        | 25 |
| 5.4.5 Test Case 5 .....                                        | 26 |
| 5.5 Simulation Log Output .....                                | 27 |
| 6. UART Integration .....                                      | 28 |
| 6.1 Command Protocol .....                                     | 28 |
| 6.2 UART Bridge Module .....                                   | 28 |
| 6.3 Data Channel vs. Control Channel Separation .....          | 29 |
| 7. Future Work .....                                           | 29 |
| 7.1 Parallel Packet Processing — Multi-Grant Crossbar .....    | 29 |
| 7.2 CPU Control Unit — Dynamic Routing Table Management .....  | 30 |
| 7.3 Link-State Routing — Dijkstra's Algorithm .....            | 30 |
| 7.4 UART Bridge Integration into Top-Level and Testbench ..... | 32 |
| 8. References .....                                            | 32 |

## 1. Introduction

---

This report documents the design and implementation of a hardware-based packet switch using Verilog Hardware Description Language (HDL). The project was developed as part of the Digital Design and FPGA Flow training program NTI . The design targets Register Transfer Level (RTL) modelling and was simulated using ModelSim.

A packet switch is a fundamental building block in modern communication networks. It receives data packets on multiple input ports, examines each packet header to determine the correct destination, and forwards the packet to the appropriate output port. Unlike circuit-switched networks, which maintain a dedicated communication path for each session, packet-switched networks share network resources dynamically and efficiently.

The project is implemented in two architectural variants. The first is a standalone packet switch that operates as a self-contained RTL system. The second extends the design to incorporate a UART (Universal Asynchronous Receiver-Transmitter) interface, enabling the switch to communicate with an external host for control-plane operations such as routing table configuration.

### 1.1 Project Objectives

- Design a 4-port packet switch capable of routing 64-bit packets based on destination address.
- Implement a standard packet header format including destination address, source address, packet type, priority, and payload length fields.
- Design and verify individual sub-modules: FIFO queue, header parser, routing table, round-robin arbiter, and crossbar switch fabric.
- Integrate all modules into a complete top-level design using structural Verilog modelling.
- Write comprehensive testbenches for each module and for the full system.
- Extend the design with a UART interface for external routing table configuration.

## 1.2 Design Tools and Language

| Item            | Detail                                |
|-----------------|---------------------------------------|
| HDL Language    | Verilog-2005                          |
| Simulation Tool | ModelSim (Mentor Graphics)            |
| Target          | RTL Simulation (FPGA-synthesisable)   |
| Clock           | Single synchronous clock domain       |
| Reset           | Active-low asynchronous reset (rst_n) |

## 2. System Architecture

---

The packet switch is constructed as a hierarchy of modular RTL blocks. Each block has a well-defined interface, a single responsibility, and is independently verifiable. The two top-level architectures — one without UART and one with UART — share the same core switching logic, differing only in how the routing table is configured.

### 2.1 Architecture 1 — Standalone Packet Switch

In this variant, the routing table is configured programmatically at simulation time through the top-level control-plane write ports (`rt_wr_en`, `rt_wr_addr`, `rt_wr_port`, `rt_wr_valid`). This is the primary design used for functional verification of the switching logic. The overall data path is as follows:

- Step 1 — Incoming packets are written into per-port input FIFOs.
- Step 2 — The header parser reads the FIFO output combinationally and extracts all header fields.
- Step 3 — The routing table performs a lookup using the destination address and returns the target output port.
- Step 4 — The round-robin arbiter selects which input port is served each clock cycle.
- Step 5 — The crossbar switch fabric connects the winning input to the correct output.
- Step 6 — The packet is written into the output FIFO for the destination port.
- Step 7 — The downstream device reads from the output FIFO when ready.

PACKET SWITCH TOP MODULE

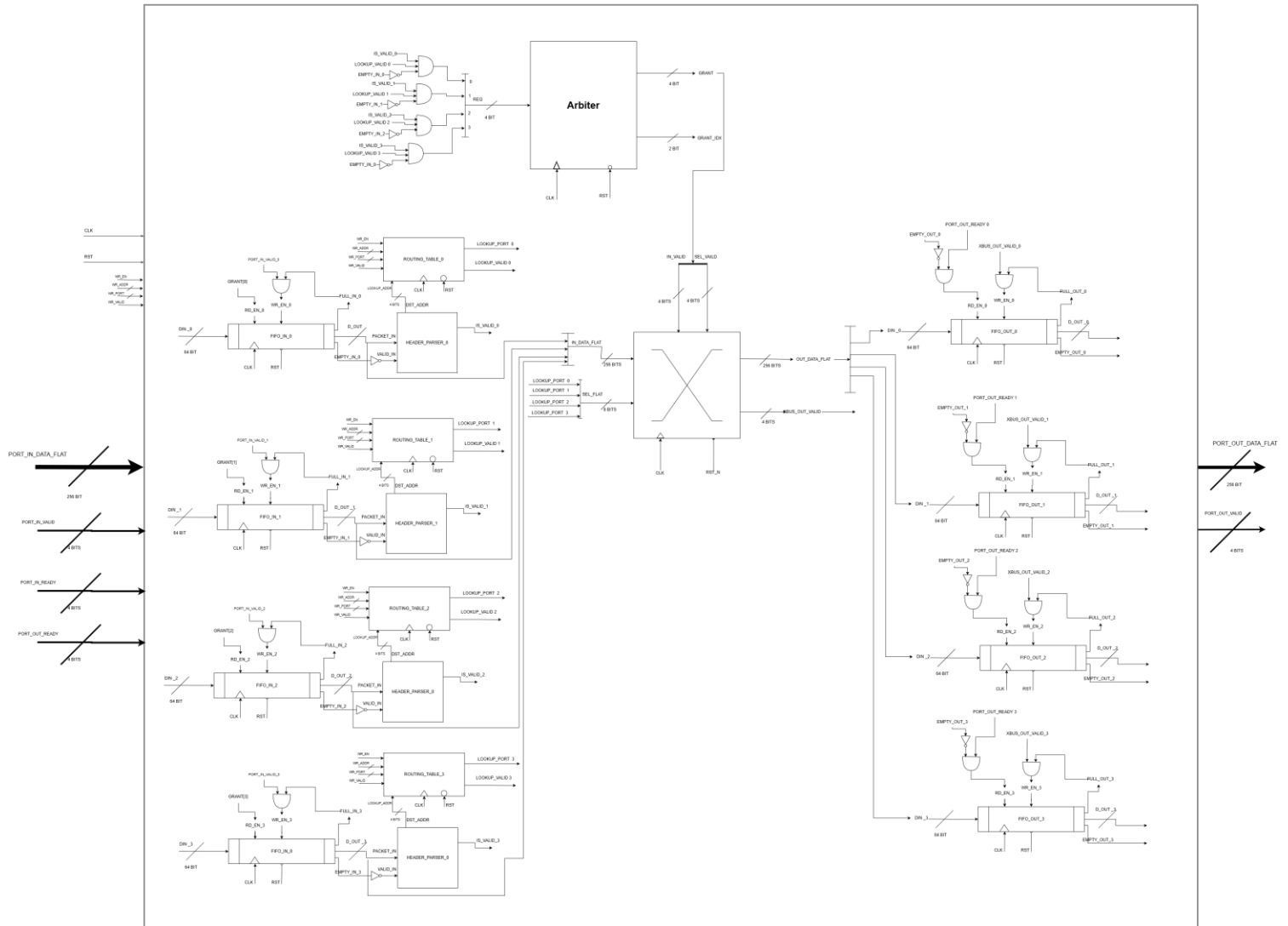


Figure 1: Top-level architecture of the standalone packet switch showing all sub-modules and their interconnections.

## 2.2 Architecture 2 — Target UART-Integrated Packet Switch

The target UART-integrated architecture extends the verified packet-switch fabric with a serial UART interface, allowing packets to be injected into and drained from the switch over a standard two-wire UART link rather than a parallel bus. The architecture uses the existing `uart_rx` / `uart_tx` cores together with purpose-built bridge modules. The individual UART and bridge modules are complete and verified; wiring the full set of instances into `packet_switch_top` and validating the complete path remain system-level integration work.

uart\_to\_packet\_bridge collects eight consecutive bytes from uart\_rx (most-significant byte first) into a single 64-bit word and presents it to one switch input port using the standard valid/ready handshake.

packet\_to\_uart\_bridge takes a 64-bit word off a switch output port and serializes it back out through uart\_tx, one byte at a time, waiting for the UART to go idle between bytes.

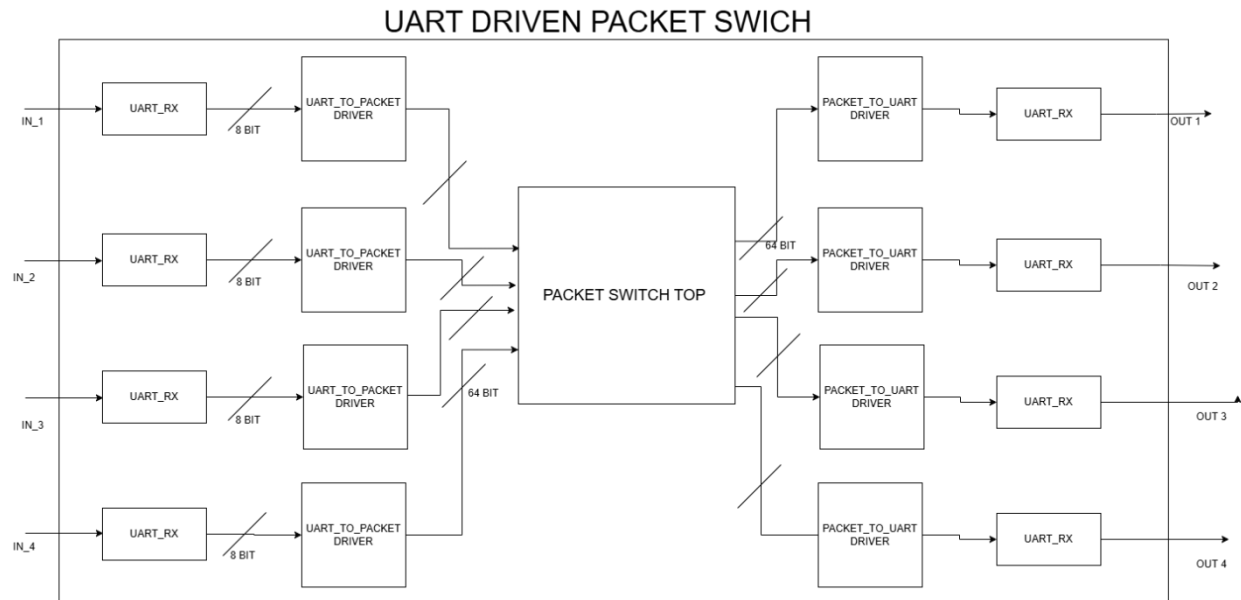


Figure 2: Extended architecture showing the UART bridge connected to packet switch TOP module

## 2.3 Key Design Decisions

| Design Decision | Choice Made                                  | Reason                                                                                                   |
|-----------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------|
| FIFO Output     | First-word fall-through (combinational dout) | Crossbar must read data on the same cycle as grant — registered output causes a one-cycle stale-data bug |
| Pointer Wrap    | Natural bit-width overflow (no modulo)       | A correctly sized register wraps automatically; % DEPTH adds redundant hardware                          |
| Port Arrays     | Flattened wide buses                         | ModelSim Verilog-2001 does not accept packed arrays as module ports                                      |



| Design Decision | Choice Made                             | Reason                                                                                  |
|-----------------|-----------------------------------------|-----------------------------------------------------------------------------------------|
| Arbiter Style   | Round-robin with registered grant       | Guarantees fairness; no port is starved regardless of traffic pattern                   |
| Reset Style     | Active-low asynchronous (negedge rst_n) | Industry standard; allows reset independent of clock                                    |
| Header          | Shared .vh define file                  | Single source of truth for all bit-field positions; prevents mismatches between modules |
| Routing Table   | RAM with valid bit per entry            | Allows distinguishing between a miss (invalid entry) and a valid route to port 0        |

### 3. Packet Format

All packets in this design are fixed-size 64-bit words. This simplification avoids variable-length packet handling and multi-cycle packet reassembly, making the design tractable as an RTL learning project while remaining structurally representative of real network packet switching. Each 64-bit word is divided into a 16-bit header region and a 48-bit payload region.

#### 3.1 Packet Field Definitions

| Field                          | Bit Range | Width   |
|--------------------------------|-----------|---------|
| dst_addr (destination address) | [63:60]   | 4 bits  |
| src_addr (source address)      | [59:56]   | 4 bits  |
| pkt_type (packet type)         | [55:54]   | 2 bits  |
| priority (service priority)    | [53:52]   | 2 bits  |
| pkt_len (payload length hint)  | [51:48]   | 4 bits  |
| payload (data content)         | [47:0]    | 48 bits |

The destination address field (dst\_addr) is the primary routing key. It is a 4-bit field that can address up to 16 distinct output ports, although the current implementation supports 4 physical ports. The routing table maps each destination address value to one of the 4 output ports.

#### 3.2 Packet Type Encoding

| pkt_type Value    | Meaning                                                    |
|-------------------|------------------------------------------------------------|
| 2'b00 — DATA      | Standard data packet — routed to destination port normally |
| 2'b01 — CONTROL   | Control-plane packet — handled by the local CPU or bridge  |
| 2'b10 — BROADCAST | Packet is forwarded to all output ports simultaneously     |

| pkt_type Value | Meaning                                            |
|----------------|----------------------------------------------------|
| 2'b11 — DROP   | Packet is discarded immediately without forwarding |

### 3.3 Priority Encoding

| priority Value   | Meaning                                                        |
|------------------|----------------------------------------------------------------|
| 2'b00 — LOW      | Best-effort delivery; served last when resources are contested |
| 2'b01 — NORMAL   | Standard priority; default for most traffic                    |
| 2'b10 — HIGH     | Elevated priority; preferred over normal and low traffic       |
| 2'b11 — CRITICAL | Highest priority; for time-sensitive or control traffic        |

The priority field is defined in the packet format but is not yet used by the arbiter in the current implementation. The current arbiter implements strict round-robin without priority weighting. Priority-aware scheduling is identified as a future enhancement in Section 7.

### 3.4 Shared Header Definition File

All bit-field positions are defined in a shared Verilog header file (packet\_defs.vh) using `define directives with include guards. This file is included by every module that needs to parse or construct packet headers. This design pattern eliminates the risk of mismatched bit positions between modules.

#### packet\_defs.vh structure:

```
`ifndef PACKET_DEFS_VH
`define PACKET_DEFS_VH
`define PKT_WIDTH    64
`define ADDR_WIDTH   4
`define DST_HI       63
`define DST_LO       60
`define PKT_DATA     2'b00
`define PKT_DROP     2'b11
// ... (all other field definitions)
`endif
```

## 4. Module Descriptions

---

The design is composed of six RTL modules, each implemented in a separate Verilog source file. The modules are connected in a strict hierarchy with the top-level module (`packet_switch_top.v`) instantiating all others. Each module is described in detail below.

### 4.1 FIFO Queue (`fifo_queue.v`)

The FIFO (First-In First-Out) queue module provides elastic buffering between the input ports and the switching fabric, and between the switching fabric and the output ports. It is the most fundamental module in the design and is reused eight times in the top level — four instances on the input side and four on the output side.

#### 4.1.1 Internal Structure

The FIFO is implemented as a circular buffer. A fixed-size memory array stores the packet words. Two pointers — a write pointer (`wr_ptr`) and a read pointer (`rd_ptr`) — track the next location to write into and the next location to read from, respectively. A count register tracks how many valid entries currently exist in the buffer, which is used to derive the full and empty status flags.

#### 4.1.2 First-Word Fall-Through

A critical design decision is that the data output (`dout`) is combinational, not registered. The `dout` signal is implemented as a continuous assignment: `assign dout = mem[rd_ptr]`. This means the head of the queue is always immediately visible on the output wire without waiting for a clock edge. This property is known as First-Word Fall-Through (FWFT) and is required to avoid a one-cycle stale-data bug in the crossbar connection.

The bug that would occur with a registered output is: the arbiter grants port N at cycle T, and simultaneously asserts `rd_en`. But because `dout` is registered, it only updates at cycle T+1 — the crossbar reads the old (stale) value at cycle T and forwards the wrong packet.

### 4.1.3 Port Interface

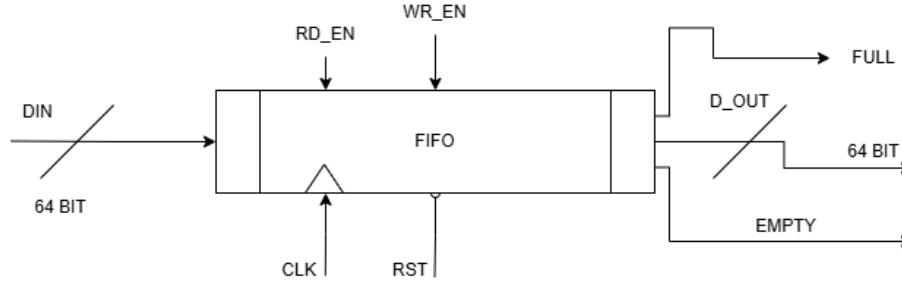


Figure 3: FIFO queue module interface showing the packet data path, read/write controls, and FIFO status signals.

| Port  | Direction     | Description                                            |
|-------|---------------|--------------------------------------------------------|
| clk   | input         | System clock — rising edge active                      |
| rst_n | input         | Active-low asynchronous reset                          |
| wr_en | input         | Write enable — write din into FIFO on next rising edge |
| din   | input         | Data input — DATA_WIDTH bits wide                      |
| rd_en | input         | Read enable — advance read pointer on next rising edge |
| dout  | output (wire) | Data output — always shows current head of queue       |
| full  | output        | 1 when FIFO is full — upstream must not write          |
| empty | output        | 1 when FIFO is empty — downstream must not read        |

### 4.1.4 Parameters

| Parameter  | Default / Description                |
|------------|--------------------------------------|
| DATA_WIDTH | 64 — width of each data word in bits |
| DEPTH      | 16 — number of storage slots         |

## 4.2 Header Parser (header\_parser.v)

The header parser is a purely combinational module. It receives the 64-bit packet word from the FIFO output and immediately extracts all header fields using Verilog bit slicing. There is no clock or sequential logic inside this module — the output updates within the same combinational propagation delay whenever the input changes. This is intentional: by the time the arbiter grants a port and the FIFO asserts its output, all routing information must already be available without any additional latency.

### 4.2.1 Bit Slicing

All field extraction is performed using constant bit-range selects, for example: `assign dst_addr = packet_in[DST_HI:DST_LO]`. The bit positions are taken from the shared `packet_defs.vh` header file, ensuring consistency with every other module that references the packet structure.

### 4.2.2 Port interface

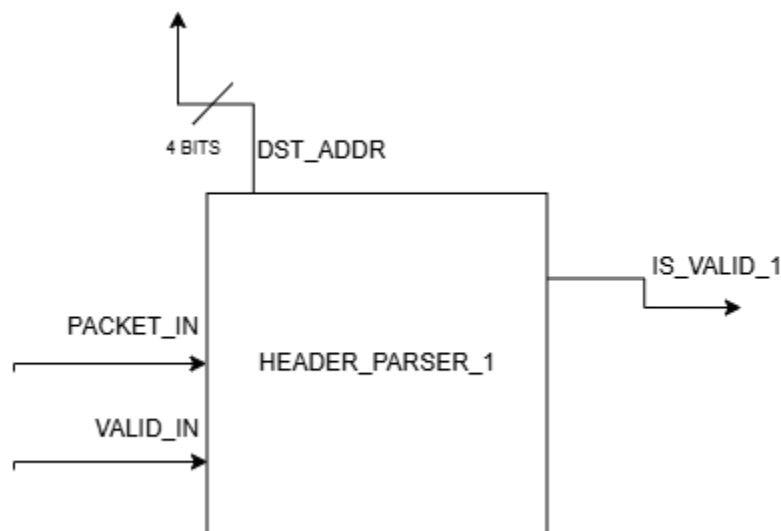


Figure 4: Header parser module port interface showing the 64-bit packet input and decoded header fields used by the routing logic.

## 4.3 Routing Table (routing\_table.v)

The routing table maps a destination address to an output port number. It is implemented as a small synchronous RAM (register array) with one write port for control-plane updates and one read port for packet forwarding lookups. The table contains 16 entries, one for each possible 4-bit destination address value.

#### 4.3.1 Entry Format

Each table entry is  $\text{PORT\_WIDTH}+1$  bits wide. The most significant bit is a valid flag indicating whether this entry contains a programmed route. The remaining  $\text{PORT\_WIDTH}$  bits hold the output port number. This packing allows both the route and its validity to be read in a single memory access. The internal memory array is named `rt_mem` to avoid a conflict with the Verilog reserved keyword 'table'.

#### 4.3.2 Important Initialisation Note

After reset, all `rt_mem` entries are zero, which means all valid flags are zero. Any packet lookup during this state returns `lookup_valid = 0`, which causes `arb_req = 0` for that port, which means the arbiter never grants it, and the packet remains permanently in the FIFO. To avoid this deadlock, routing table entries must be written before any packets are sent. In simulation this is done in the testbench immediately after reset deassertion.

| Address          | Ports |
|------------------|-------|
| ADDR0 TO ADDR3   | 0     |
| ADDR4 TO ADDR7   | 1     |
| ADDR8 TO ADDR11  | 2     |
| ADDR12 TO ADDR15 | 3     |

### 4.3.3 Port interface

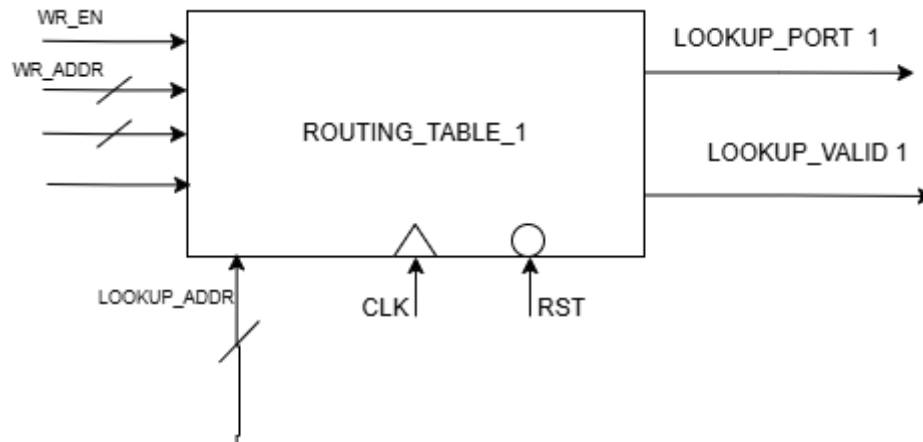


Figure 5: ModelSim memory viewer showing the routing-table entries after testbench initialisation, including address, valid bit, and mapped output port.

## 4.4 Round-Robin Arbiter (arbiter.v)

The arbiter is the traffic manager of the switch. Each clock cycle it receives a request vector (req) where each bit represents whether the corresponding input port has a valid packet ready to forward. It selects exactly one winning port and asserts a one-hot grant signal, which drives both the FIFO read enable and the crossbar select input.

### 4.4.1 Round-Robin Algorithm

The arbiter implements a round-robin scheduling policy. A last\_served register remembers which port was granted in the most recent active cycle. The combinational scan logic iterates through ports starting from (last\_served + 1) modulo NUM\_PORTS. The first port found with a pending request wins. This guarantees that no port is permanently skipped — a property called starvation freedom.

### 4.4.2 One-Hot Grant

The grant output is one-hot encoded: exactly one bit is set to indicate the winning port. For example, grant = 4'b0100 means port 2 won this cycle. The grant\_idx output provides the same information in



binary format ( $\text{grant\_idx} = 2'b10$  for the same example). The crossbar uses  $\text{grant}$  as the select signal for each output multiplexer.

#### 4.4.3 Step-by-Step Operation

- Combinational block computes  $\text{next\_grant}$  by scanning from  $(\text{last\_served}+1)$ .
- A found flag stops the scan after the first requesting port is identified.
- Sequential block registers  $\text{next\_grant}$  into  $\text{grant}$  on the rising clock edge.
- If any bit of  $\text{next\_grant}$  is set (reduction-OR),  $\text{last\_served}$  is updated to the winner index.
- If no port is requesting,  $\text{grant}$  stays zero and  $\text{last\_served}$  is unchanged.

#### 4.4.4 Port Interface

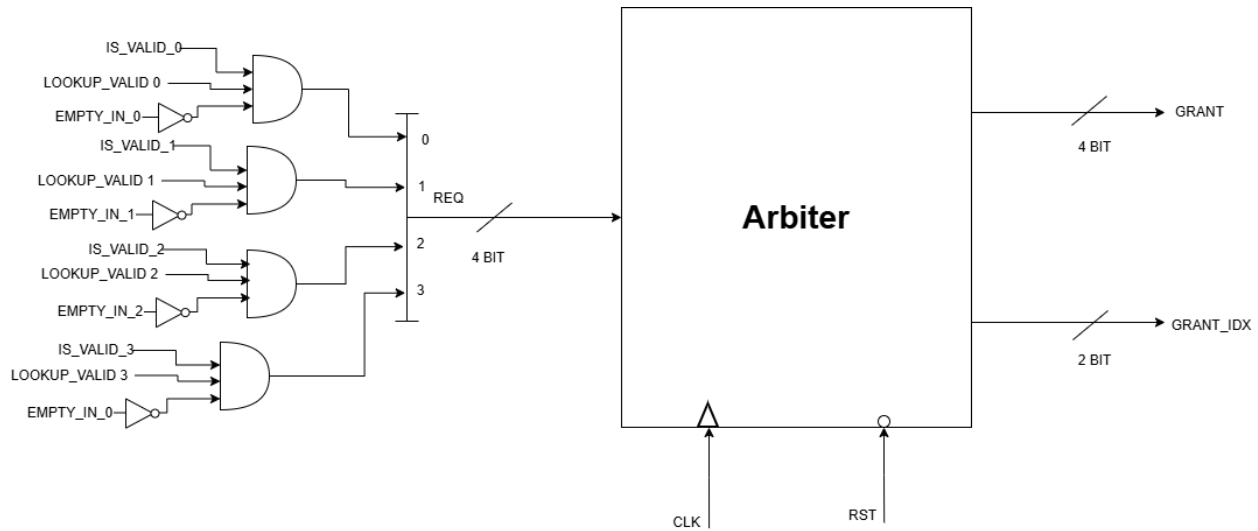


Figure 6: Round-robin arbiter interface showing the request inputs, one-hot grant output, and selected port index.

| Port  | Direction              | Description                                |
|-------|------------------------|--------------------------------------------|
| clk   | input                  | System clock                               |
| rst_n | input                  | Active-low asynchronous reset              |
| req   | input [NUM_PORTS-1:0]  | Request vector — one bit per port          |
| grant | output [NUM_PORTS-1:0] | One-hot grant — one bit set for the winner |

| Port      | Direction                                  | Description                      |
|-----------|--------------------------------------------|----------------------------------|
| grant_idx | output [ $\log_2(\text{NUM\_PORTS})-1:0$ ] | Binary index of the winning port |

## 4.5 Crossbar Switch Fabric (crossbar.v)

The crossbar is the physical connection engine of the switch. It implements a full 4x4 non-blocking crossbar fabric, meaning any input port can be connected to any output port simultaneously, provided no two inputs are routed to the same output in the same cycle (which the arbiter prevents by granting only one port per cycle in the current implementation).

The crossbar receives the packet data from the input FIFOs, the grant vector from the arbiter, and the routing decision (sel) from the routing table. For each input port  $i$  that has an active grant, it routes  $\text{in\_data}[i]$  to  $\text{out\_data}[\text{sel}[i]]$ .

### 4.5.1 Flat Bus Interface

Because ModelSim's Verilog-2001 mode does not accept packed array ports, all multi-port data signals are implemented as wide flat buses. For example, instead of  $\text{in\_data}[0:\text{NUM\_PORTS}-1]$  where each element is  $\text{DATA\_WIDTH}$  bits, the port is declared as  $\text{in\_data}[(\text{NUM\_PORTS}*\text{DATA\_WIDTH})-1:0]$ . Inside the module, individual port data is accessed by slicing: port  $N$  occupies bits  $[(N+1)*\text{DATA\_WIDTH}-1 : N*\text{DATA\_WIDTH}]$ .

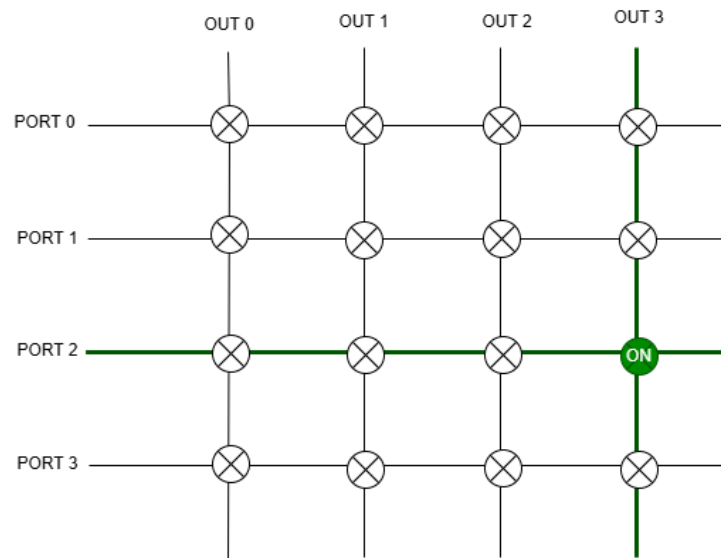


Figure 7: 4x4 crossbar fabric showing the active connection matrix. The highlighted row/column intersection represents the selected switch point for the current transfer.

#### 4.5.2 Port Interface

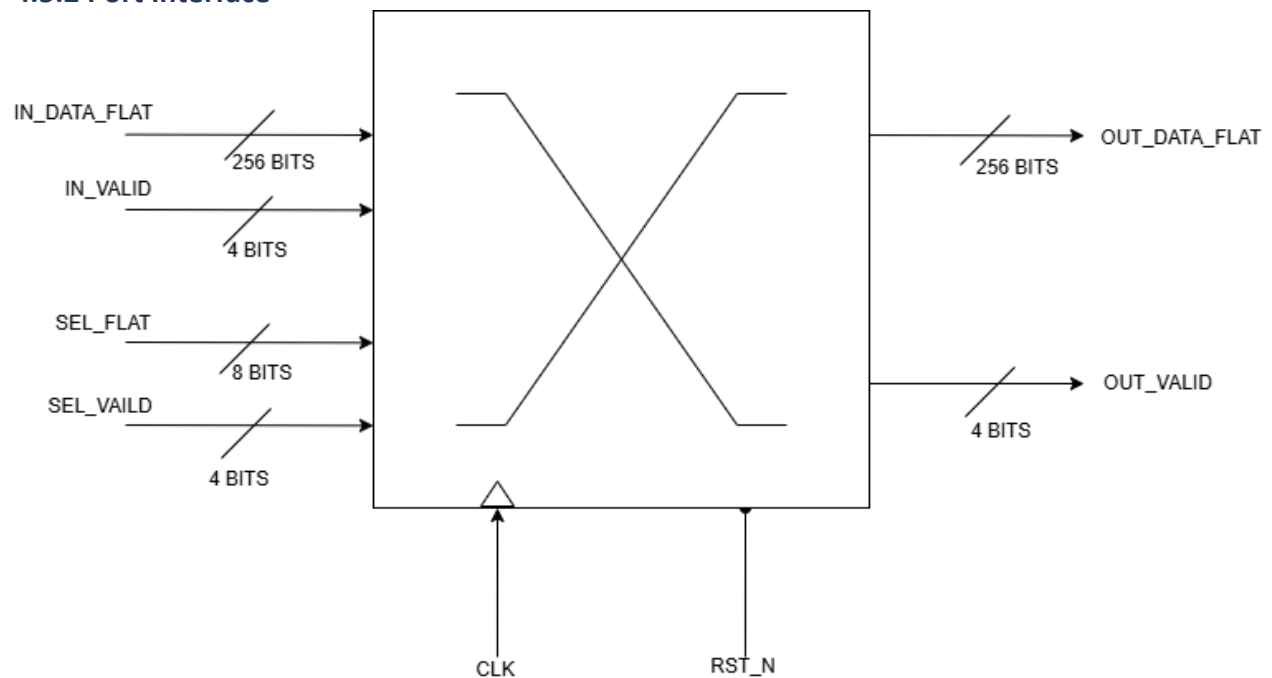


Figure 8: Crossbar switch interface illustrating the 4x4 switching fabric and its flattened input/output bus connections.

#### 4.5.3 Default Output Clearing

At the start of every clock cycle, all output data and output valid signals are cleared to zero by default. Only then are the active connections asserted. This ensures that output ports with no incoming packet in a given cycle present a clean zero rather than stale data from a previous cycle.

## 4.6 Top-Level Module (packet\_switch\_top.v)

The top-level module is a structural module — it contains no combinational or sequential logic of its own. Its sole purpose is to instantiate all sub-modules and connect them together using internal wires. This strict separation between structural and behavioural code makes the hierarchy clear and allows each sub-module to be swapped or upgraded independently.

### 4.6.1 Generate Blocks

The four input FIFOs, four header parsers, four routing table instances, and four output FIFOs are instantiated using Verilog generate-for loops. This avoids writing four copies of each instantiation and ensures that adding more ports only requires changing the NUM\_PORTS parameter rather than adding more code.

### 4.6.2 Back-Pressure

The top-level implements flow control on both the input and output sides. On the input side, port\_in\_ready is asserted low whenever the corresponding input FIFO is full, signalling the upstream device to pause. On the output side, the output FIFO only forwards to the external port when port\_out\_ready is asserted by the downstream device.

### 4.6.3 Flat Bus Wiring

All crossbar interconnects inside the top-level use the flattened wide-bus convention. The generate blocks use localparam slice calculations to connect the correct bit ranges from each FIFO output to the correct slice of the crossbar input bus, and similarly on the output side.

## 5. Testbench and Verification

Functional verification was performed using directed testbenches written in Verilog. Each sub-module has a dedicated testbench that verifies its behaviour in isolation. A system-level testbench verifies the integrated top-level design. All testbenches were simulated in ModelSim and produce pass/fail output through \$display statements.

### 5.1 Testbench Strategy

The verification approach follows a bottom-up strategy. Each module is verified independently before being integrated into the system-level testbench. This makes it straightforward to localise and fix bugs — a failure in the system testbench that also reproduces in the unit testbench is a module bug, while a failure that only appears in the system testbench is likely a wiring or timing issue at the integration level.

| Testbench File         | What It Verifies                                                                    |
|------------------------|-------------------------------------------------------------------------------------|
| tb_fifo_queue.v        | Write, read, full flag, empty flag, wrap-around, simultaneous read+write            |
| tb_header_parser.v     | Correct field extraction for all packet types and edge-case bit patterns            |
| tb_routing_table.v     | Write entries, lookup, invalid entry behaviour, reset clearing                      |
| tb_arbiter.v           | Reset, single request, round-robin order, starvation freedom, one-hot grant         |
| tb_crossbar.v          | Correct routing to each output, default clearing, simultaneous grants               |
| tb_packet_switch_top.v | End-to-end packet flow, routing table initialisation, back-pressure, multiple ports |

### 5.2 Packet Switch Testbench Details

All module-level testbenches were completed successfully. The corresponding testbench files are available in the repository under the tb sub-modules directory.

The top-level packet-switch testbench focuses on five directed routing scenarios that exercise individual destinations and simultaneous requests.

For test cases 1–4, a packet is injected on input port 0 with a different destination address and the observed output port is compared with the routing-table expectation. Test case 5 injects two packets simultaneously to verify round-robin arbitration and correct delivery without packet loss.

| Test | Destination                                                  | Expected output  |
|------|--------------------------------------------------------------|------------------|
| 1    | 2                                                            | Port 0           |
| 2    | 6                                                            | Port 1           |
| 3    | 9                                                            | Port 2           |
| 4    | 14                                                           | Port 3           |
| 5    | Input port 1 → destination 5<br>Input port 3 → destination 1 | Port 1<br>Port 3 |

### 5.3 Known Bugs Identified and Fixed During Verification

Verification exposed several integration-sensitive issues. The fixes below were applied before the final directed tests were run.

- FIFO output timing: the FIFO uses first-word fall-through (combinational dout) so the crossbar sees the current head word in the same cycle as the arbiter grant; a registered dout would create a one-cycle stale-data error.
- Routing-table validity: each entry includes a valid bit so an uninitialised lookup can be distinguished from a valid route to output port 0.
- Arbiter fairness: the round-robin pointer advances only when a request is granted, preventing starvation while keeping the grant one-hot.

### 5.4 Simulation Waveforms

The following ModelSim captures document the top-level routing behaviour. Each input waveform shows packet injection and the corresponding valid handshake; the following output waveform shows the selected destination port and the packet data after the routing and arbitration latency.

#### 5.4.1 Test Case 1

Inject the first packet into input port 0. The packet uses destination address 2, which the routing table maps to output port 0.



### 5.4.2 Test Case 2

The timing diagram displays several digital signals over time. The signals are:

- dk**: A clock signal represented by a green square wave.
- port\_in\_data\_flat**: An input data bus shown as a horizontal yellow line.
- port\_in\_ready**, **port\_out\_ready**: Ready signals shown as horizontal blue lines.
- port\_in\_valid**, **port\_out\_valid**: Valid signals shown as horizontal red lines.
- rst\_n**: A reset signal shown as a horizontal purple line.

A yellow vertical line indicates the current simulation time at 79835 ps. Below the diagram, the text "sim:/tb\_packet\_switch\_top/rst\_n @ 79835 ps" is displayed.

Figure 11: Test case 2 — packet with destination address 6 inserted into input port 0 (port\_in\_valid = 0001).

After six clock cycles, the packet appears at output port 1, confirming correct routing for destination address 6.

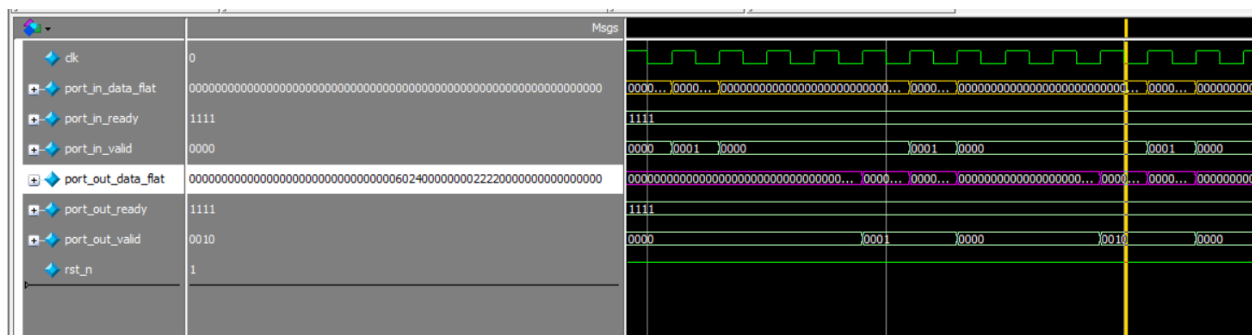


Figure 12: Test case 2 — after six clock cycles, the packet is correctly forwarded to output port 1 (port\_out\_valid = 0010).

### 5.4.3 Test Case 3

Inject the third packet into input port 0. The packet uses destination address 9, which the routing table maps to output port 2.

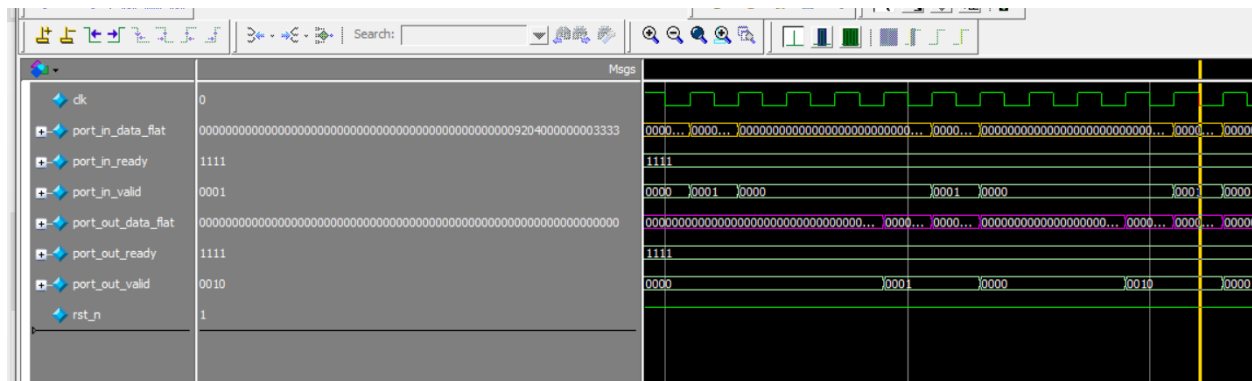


Figure 13: Test case 3 — packet with destination address 9 inserted into input port 0 (port\_in\_valid = 0001).

After six clock cycles, the packet appears at output port 2, confirming correct routing for destination address 9.





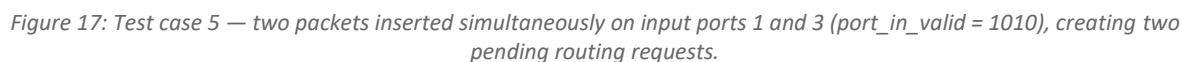
Inject the fourth packet into input port 0. The packet uses destination address 14, which the routing table maps to output port 3.



Page 25 of 32



Inject two packets simultaneously on input ports 1 and 3. This scenario verifies that the round-robin arbiter services both requests in successive cycles without losing either packet.



After six clock cycles, the first packet appears at output port 1, confirming the first arbitration decision.



After a further six clock cycles, the second packet appears at output port 3, confirming that the arbiter eventually services the second pending request.



Figure 19: Test case 5 — after a further six clock cycles, the second packet is correctly forwarded to output port 3 (port\_out\_valid = 1000), confirming both requests are serviced without loss.

## 5.5 Simulation Log Output

The simulation transcript below provides a textual cross-check of the waveform results. The \$display messages report the routed packet values and destination ports, followed by the testbench completion status.

```
/SIM 22> run -all
# =====
# Starting Packet Switch Test
# =====
# [PASS] Packet 0x2114000000001111 routed to output port 0
# [PASS] Packet 0x6024000000002222 routed to output port 1
# [PASS] Packet 0x9204000000003333 routed to output port 2
# [PASS] Packet 0xe334000000004444 routed to output port 3
# [PASS] Input 1 packet routed to output 1
# [PASS] Input 3 packet routed to output 0
# =====
# Testbench Finished
# =====
# ** Note: $finish : D:/embeddedprojectgithub/Package-Switch-Using-Verilog-/top_tb.v(474)
# Time: 326 ns Iteration: 0 Instance: /tb_packet_switch_top
# 1
```

Figure 20: ModelSim transcript window showing the \$display PASS/FAIL output generated by the top-level packet-switch testbench.

## 6. UART Integration

---

The UART support is complete at the module level: `uart_rx`, `uart_tx`, `uart_to_packet_bridge`, and `packet_to_uart_bridge` are individually complete and verified. The remaining work is system-level integration—connecting four receive paths and four transmit paths to `packet_switch_top` and validating the complete UART-driven switch with a dedicated testbench.

### 6.1 UART Bridge Module

The planned `uart_packet_switch_top` wrapper connects the packet switch fabric to independent UART channels on all four ports. The bridge modules convert between the serial byte stream and the switch's native 64-bit packet interface.

- `uart_to_packet_bridge` collects eight consecutive bytes from `uart_rx` (most-significant byte first), assembles them into one 64-bit word, and presents the word to the corresponding switch input using the standard valid/ready handshake.
- `packet_to_uart_bridge` accepts a 64-bit word from the corresponding switch output and serialises it through `uart_tx` one byte at a time, waiting for the UART transmitter to become idle between bytes.
- Four `uart_rx` / `uart_to_packet_bridge` pairs are required for the four input ports, and four `packet_to_uart_bridge` / `uart_tx` pairs are required for the four output ports so that each port can operate independently.

## 7. Future Work

---

The current implementation represents a functionally correct baseline packet switch. Several significant enhancements are planned for future versions of the design. These enhancements progress from improved hardware utilisation to full distributed routing intelligence.

### 7.1 Parallel Packet Processing — Multi-Grant Crossbar

The current arbiter grants exactly one input port per clock cycle. This means only one packet moves through the switch fabric per cycle, even when multiple input ports have packets destined for different output ports that do not share any resources in the crossbar. This is a significant underutilisation of the non-blocking crossbar fabric.

The planned enhancement will implement a parallel matching algorithm — similar to the iSLIP algorithm used in real-world switches — that identifies sets of (input, output) pairs with no conflicts and grants all of them simultaneously in the same cycle. For example, if port 0 wants to send to output 1 and port 2 wants to send to output 3, both can proceed simultaneously because they use different output ports. This enhancement can achieve up to 4x throughput improvement when traffic is uniformly distributed.

- Implement a conflict-detection matrix that checks whether any two pending requests share an output port.
- Modify the arbiter to produce a multi-hot grant vector where multiple bits can be set simultaneously.
- Verify the crossbar correctly handles simultaneous grants with no output conflict.
- Add a maximal matching algorithm (iSLIP or PIM — Parallel Iterative Matching) for optimal throughput.

## 7.2 CPU Control Unit — Dynamic Routing Table Management

The current design updates the routing table through direct port-level signals from the testbench or the UART bridge. A future enhancement will integrate a small embedded CPU core or a custom finite-state-machine controller that manages the routing table autonomously.

This CPU control unit will be responsible for:

- Sending and receiving HELLO messages on all ports at regular intervals to discover active neighbours.
- Maintaining a neighbour adjacency table listing all directly connected nodes and their current link states.
- Detecting when a neighbour becomes unreachable (HELLO timeout) and marking its routes as invalid.
- Writing new routing table entries and clearing stale entries through the `rt_wr` port.
- Responding to control-plane queries from the UART interface.

### 7.3 Link-State Routing — Dijkstra's Algorithm

The highest-complexity planned enhancement is the implementation of a link-state routing protocol inspired by OSPF (Open Shortest Path First). Each switch will maintain a complete map of the network topology and compute the shortest path to every destination using Dijkstra's algorithm.

The implementation plan is as follows:

- Each switch node broadcasts Link-State Advertisement (LSA) packets on all active ports, containing its node ID and a list of its directly connected neighbours with associated link costs (initially based on hop count, later extended to bandwidth or latency metrics).
- Received LSAs are flooded throughout the network so every switch builds the same global topology graph.
- Each switch runs Dijkstra's shortest-path algorithm on the stored topology graph to compute the next-hop port for every possible destination address.
- The computed routes are written into the routing table, replacing any previous entries.
- When a link-state change is detected (neighbour goes down or cost changes), a new LSA is generated and the routing computation is re-run.

Implementing Dijkstra's algorithm in RTL is non-trivial and requires a multi-cycle iterative hardware implementation. The graph adjacency structure will be stored in a small distributed RAM, and the algorithm will execute over multiple clock cycles using a controller FSM that iterates over graph nodes, maintaining a visited set and a tentative distance table.

An alternative simpler approach for an initial implementation is the Bellman-Ford algorithm, which is more naturally expressed as an iterative RTL loop and does not require a priority queue data structure. Bellman-Ford converges in at most  $N-1$  iterations for an  $N$ -node network and handles negative-cost edges, though Dijkstra is preferred for its better average-case performance in non-negative-cost networks.

## 7.4 UART Bridge Integration into Top-Level and Testbench

As noted in Section 6, the UART receive/transmit cores and both bridge module types are individually complete and verified, but the UART-driven architecture has not yet been wired into `packet_switch_top` or exercised by a dedicated system-level testbench. The remaining implementation steps are:

- Instantiate four `uart_rx` / `uart_to_packet_bridge` pairs and four `packet_to_uart_bridge` / `uart_tx` pairs inside `uart_packet_switch_top`, connected to the existing `packet_switch_top` input and output ports.
- Implement the CONTROL-packet interception path described in Section 6.1 so routing-table configuration commands are diverted before reaching the crossbar.
- Write `tb_uart_packet_switch_top.v` to drive byte streams into each `uart_rx` and verify the corresponding bytes at each `uart_tx`, using the same pass/fail `$display` style as `tb_packet_switch_top.v`.
- Verify end-to-end back-pressure: when an input FIFO becomes full, the condition must propagate to `uart_to_packet_bridge` so that byte reception stalls cleanly rather than dropping data.

## 7.5 Command Protocol

The UART link carries both data-plane packets and control-plane commands over the same physical byte stream. The existing `pkt_type` field (Section 3.2) distinguishes the two roles without introducing a separate command channel.

- DATA packets (`pkt_type = 2'b00`) received over UART are treated like packets injected through the parallel interface: they enter the input queue, are parsed and routed, and are forwarded to the selected output port.
- CONTROL packets (`pkt_type = 2'b01`) are intercepted before the crossbar and interpreted as configuration commands, primarily routing-table writes using `rt_wr_en`, `rt_wr_addr`, and `rt_wr_port`.
- BROADCAST and DROP packet types retain the behaviour defined in Section 3.2 regardless of whether the packet originated from the parallel interface or the UART link.
- The 64-bit packet word remains the common data and control container, allowing `packet_defs.vh`, the header parser, and routing-table logic to be reused without creating a second packet format.

## 8. References

---

| Closest Reference Papers — Fill in Full Citations                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [1] B. Prabhakar and N. McKeown, 'On the Speedup Required for Combined Input Output Queued Switches,' IEEE Transactions on Automatic Control, 1999.                |
| [2] Variable_Packet-Size_Buffered_Crossbar_Switch_Chip. <a href="#">link</a>                                                                                       |
| [2] Jehangir Rashid Dar 'FPGA Design of 8 bit 4x4 Crossbar Switch for Multi Processor System on Chip Using Round Robin Arbitration Algorithm'                      |
| [3] E. W. Dijkstra, 'A Note on Two Problems in Connexion with Graphs,' Numerische Mathematik, 1959. — The original Dijkstra shortest-path paper (Future Work 7.3). |